Session 23 Teamwork — Project Detail: a second real nested route

Context

Work in your team (six teams of four students). LIVE already built two real pages: the Workspaces List (`/workspaces`) and Workspace Detail (`/workspaces/[id]`), combining a real `GET /workspaces/:id` call with a client/server-filtered `GET /projects` call for the real project list. Your job: a second real page, one real level deeper, for a genuinely different real reason.

Goal

A real page at `/workspaces/[id]/projects/[projectId]` renders one real project's own key/name/description, plus its own real issue list -- each real issue linking into `/workspaces/[id]/projects/[projectId]/issues/[issueId]` (this session's own real homework; the link is real even before that page exists).

What to build

- A real, empty dynamic route at the real nested path above, proving `params` resolves both real ids correctly (the identical real `await params` pattern as Workspace Detail, now with two real values instead of one).
- The real fetch + not-found guard: `GET /projects/:projectId` for the project, then the real issue list.
- **The real transfer question:** `GET /issues` already supports a real `?projectId=` filter (check `api/src/issues/dto/find-issues-query.dto.ts` yourself, in the real backend repo, to confirm this). Workspace Detail had to fetch every real project and filter them itself, because `GET /projects` has no such filter. Do NOT copy that workaround here out of habit -- use the real filter the backend already gives you.
- The real render: project key/name/description, then a real linked list of its own real issues.
- The identical real `loading.tsx` / `not-found.tsx` / `error.tsx` trio LIVE already built for Workspace Detail, applied to this new segment.

Rules

Allowed: everything already shown LIVE this session (`apiFetch` , the real `Workspace` / `Project` types, the real dynamic-route/ `async params` pattern, the real `loading` / `not-found` / `error` file conventions), plus the real official Next.js documentation for dynamic routes and `fetch` .

Not yet: no Tailwind (Session 24's own real theme -- this page stays unstyled, like every other real page so far). No client-side state, no `"use client"` anywhere except `error.tsx` (structurally required there, nowhere else). No auth, no token (Session 25) -- `GET /projects/:id` and `GET /issues` are both genuinely public; do not add a fake header pretending otherwise.

Test it

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
curl -s http://localhost:3001/workspaces/1/projects/1
curl -s http://localhost:3001/workspaces/1/projects/999999
npx tsc --noEmit
```

Done when

- A real existing project shows its own real key/name/description and its own real issue list, each linking into the (not-yet-built) Issue Detail route.
- A real, genuinely missing project id shows your own real `not-found.tsx` text, not a crash.
- `GET /issues` is called WITH the real `?projectId=` filter -- not fetched in full and filtered client-side.
- `npx tsc --noEmit` exits clean.

Working time

About 38 minutes total (TW-01..09).